Мой подход: экономный ИИ-агент для одноплатного компьютера

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.

> **Идея.** На слабой плате дорого всё: память, процессорное время, каждое пробуждение, каждый сетевой запрос, а с внешней моделью — ещё и каждый токен. Поэтому агент устроен так, чтобы **как можно реже и как можно короче** обращаться к модели и при этом не пропускать важное: локально — всё, что можно решить быстро и однозначно; в облако — только сжатая справка.

1. Состав системы

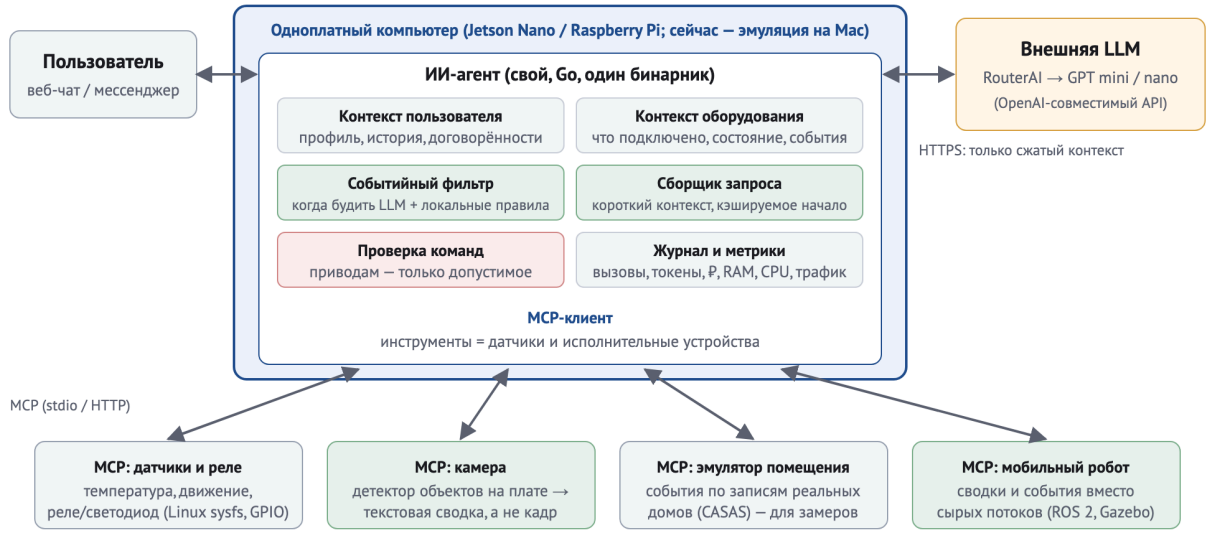


Рисунок 1 — Состав системы. Зелёным отмечены механизмы, которые сокращают вызовы модели и объём передаваемых данных; красным — проверка команд исполнительным устройствам (на схеме она показана в составе агента, а реализована в MCP-сервере оборудования, между моделью и приводами).

- **Агент** (рабочее название EdgeCtx) — собственная программа на Go: один статический исполняемый файл без зависимостей (сейчас около 13,5 МБ). Проверен в виртуальной плате (Ubuntu 24.04, ARM); по требованиям Go должен запускаться и на Raspberry Pi 5, и на устаревшей системе Jetson Nano (Ubuntu 18.04, ядро 4.9), где аналоги на Node.js штатно не запускаются. На реальных платах это ещё предстоит проверить.
- **Внешняя языковая модель** — GPT mini/nano через RouterAI (OpenAI-совместимый интерфейс). Для разработки и ускоренных прогонов — модель-заглушка с «правильными ответами», чтобы бесплатно измерять затраты политик агента.
- **MCP-серверы оборудования**: датчики и реле (описание оборудования строится автоматически из данных ядра Linux); камера с детектором объектов на плате (запланирована, ещё не написана); эмулятор помещения по записям реальных домов; компактный адаптер мобильного робота.

2. Механизмы, которые делают агента экономным

Многие приёмы по отдельности уже встречаются у аналогов (см. «Что я изучил», раздел 2) и за новые не выдаются. Новое — их сочетание в одном агенте для одноплатника, MCP-сервер, который сам строит описание оборудования по данным ядра Linux, и измерение того, при какой нагрузке каждый приём окупается.

Механизм	Что делает	Что экономит
Событийный фильтр и локальные правила	Решает, будить ли модель: пороги, подавление дребезга, объединение событий, «конец эпизода». Правила, которые модель один раз создала («если дверь открыта после 22:00 — напомни»), дальше исполняются на плате без модели	вызовы модели, токены, пробуждения процессора
Два контекста	Контекст пользователя: факты-предпочтения, история с политиками сокращения (окно, скрытие старых результатов инструментов, сводка). Контекст оборудования: перечень устройств — кратко в постоянной части запроса, состояние — снимком, изменениями или по требованию	входные токены, рубли
Стабильное начало запроса	Неизменная часть (инструкция, описания инструментов, перечень устройств) всегда идёт первой и сериализуется байт в байт одинаково; всё изменчивое — в конце	стоимость за счёт кэша провайдера (кэшированные токены в 2–10 раз дешевле; работает ли кэш через RouterAI — проверяется первым же ключом)
Компактный каталог инструментов	Несколько общих инструментов («список устройств», «прочитать свойство», «записать свойство») вместо отдельного инструмента на каждое свойство	входные токены в каждом запросе
Сводки вместо сырых данных	Камера: детектор на плате → «в комнате 1 человек». Робот: «ближайшее препятствие 0,6 м спереди» вместо полного скана лидара (около 4,4 тыс. токенов)	токены, трафик, время ответа
Асинхронные долгие действия	«Поехать к двери» возвращает управление сразу, а результат приходит событием «доехал / прервано / препятствие» — без опроса и без потока промежуточных статусов	время хода агента (он не блокируется на 16–90 с) и трафик (без потока промежуточных статусов); цена — один дополнительный вызов модели на отчёт о прибытии, если пользователь его ждёт

Детерминированная проверка команд	Каждая команда приводу проходит проверку в MCP-сервере: наличие в реестре, диапазон, свежесть состояния, блокировки, частота, подтверждение необратимых действий; при потере связи — безопасное состояние	безопасность без лишних вызовов модели; около 0,5 мкс на проверку
Учёт собственных ресурсов	Агент ведёт суточный бюджет (вызовы и входные токены): после 80 % лимита отклоняет фоновые задачи, при исчерпании обслуживает только пользователя. Состояние платы (память, процессор, температуру) модель может запросить инструментом board_health сервера оборудования; автоматического урезания по нагрузке платы пока нет	предсказуемый расход за день
Лёгкая реализация	Статический исполняемый файл на Go, одно постоянное соединение с моделью, без потоковой передачи ответов, без лишних потоков и таймеров; все таймеры работают от общих часов (это позволяет проигрывать день ускоренно)	память, пробуждения, энергия

3. Два сценария

- **«Умное помещение / лаборатория».** Датчики температуры, движения, открытия двери и освещённости; реле (свет, вентилятор); камера. Рабочий день собирается из реальных данных: частота команд пользователя — по многомесячному исследованию голосовых ассистентов, события датчиков — по записям реальных домов CASAS. В сценарий дня входят значимые ситуации («дверь открыта ночью», «перегрев»), которые агент должен заметить сам, и вопросы о прошлом («когда в последний раз открывали дверь?»).
- **«Мобильный робот».** TurtleBot3 с камерой и лидаром в симуляторе Gazebo под ROS 2, навигация Nav2. Обход точек по расписанию (локально, без модели на каждом шаге), команды пользователя («подъезжай к двери», «что в коридоре?»), события («препятствие», «человек в кадре», «низкий заряд»). Симулятор работает в отдельной виртуальной машине, агент — на «плате», связь по сети, как при реальном развёртывании.

4. Как проверяется эффективность

- **Рабочий день.** Один и тот же день проигрывается на разработанном агенте, на его вариантах с отключёнными механизмами (чтобы оценить вклад каждого), на агенте «только с периодической фоновой проверкой» (так штатно работают аналоги) и на PicoClaw в лучшей штатной настройке; nanobot в штатной настройке сравнивается по фоновому потреблению и памяти, а также в ускоренных прогонах. Для робота — сравнение своего адаптера с готовым ros-mcp-server на одном и том же агенте.

- **Метрики за день:** вызовы модели; входные, выходные и кэшированные токены; рубли; трафик; задержка «событие → действие»; память и процессор платы во времени (в том числе рост памяти за 8 часов); энергопотребление (при наличии датчика мощности); полнота реакции на значимые события; число небезопасных команд (должно быть 0).
- **Ускоренный режим.** У агента виртуальные часы, поэтому 16-часовой день проигрывается за минуты; для аналогов, у которых виртуальных часов нет, время дня сжимается в *k* раз с соответствующим пересчётом таймеров. Для каждой метрики указано, корректна ли она при сжатии (вызовы и токены — да, если все периоды аналога пересчитываются точно; процессор в сутки — нет). Например, при сжатии в 60 раз у nanobot выключилось фоновое сжатие истории, поэтому для него берётся сжатие в 15 раз. Итоговые цифры будут подтверждены реальными 8-часовыми прогонами на плате.
- **Лестница ограничений.** Тот же день — при всё более жёстких лимитах виртуальной платы: память 1024 → 512 → 256 → 128 → 64 → 32 МиБ, процессор 4 ядра → 1 ядро → 50 → 25 → 10 % ядра, сеть от локальной до 3G/EDGE с потерями и обрывами, медленная карта памяти. Результат — «карта выживаемости»: где какой агент ещё работает и как растут задержка и затраты. Ядро ноутбука в несколько раз быстрее ядра Jetson Nano, поэтому ступени по процессору калибруются по плате.
- **Статистика:** парные доверительные интервалы по трассам дня, проверка «не хуже» с допуском, пороги фиксируются до прогонов.

5. Почему это хорошо работает на одноплатнике

1. Плата делает только то, что у неё получается быстро и дёшево (фильтрация событий, правила, проверка команд, распознавание объектов), а дорогое и медленное — «рассуждение» — вызывается редко и с короткой справкой.
2. Сам агент почти не потребляет ресурсов: в простое около 13 МиБ памяти и единичные пробуждения в минуту, поэтому плата остаётся свободной для датчиков, камеры и детектора.
3. Расход на модель предсказуем и ограничен бюджетом; агент и так передаёт мало данных, поэтому плохая сеть сказывается на нём меньше, а локальные правила продолжают работать и без связи.
4. Один и тот же исполняемый файл переносится между платами без установки зависимостей — это будет проверено на старом Jetson Nano и в виртуальной плате с жёсткими лимитами (серверу оборудования для GPIO на ядре 4.9 ещё нужна поддержка старого интерфейса ядра).

Первые замеры приведены в документе «Что уже сделано и первые замеры»; по событийному фильтру и камере пока есть только расчётные ориентиры (план ВКР, гипотезы H1 и H5). Итоговые результаты будут получены по протоколу экспериментов.